
Collecte et Préparation

d'un Dataset d'Images

pour Classification d'Oiseaux

Rapport complet

Méthodologie, statistiques, limites, éthique

Repository : github.com/souhayle98/projet_scraping2

Branches : main (production) dev (développement)

Date : 19 mai 2026



de la collecte à la préparation finale

Table des matières

1	Introduction et Contexte	2
2	Méthodologie Complète	2
2.1	Étape 1 — Collecte (Scraping)	2
2.2	Étape 2 — Nettoyage (nettoyage.py)	2
2.3	Étape 3 — Prétraitement (pretraitement.py)	2
2.4	Étape 4 — Augmentation (augmentation.py)	3
2.5	Étape 5 — Division (division_dataset.py)	3
3	Difficultés Rencontrées et Solutions	3
4	Statistiques Détaillées (avec Graphiques)	3
4.1	Bilan global du pipeline	4
4.2	Graphique 1 — Images collectées vs cible (100) par espèce	4
4.3	Graphique 2 — Suppressions par motif (camembert)	5
4.4	Graphique 3 — Avant / après augmentation	5
4.5	Graphique 4 — Répartition finale train/val/test (camembert global) . .	6
5	Analyse Critique des Limites et Biais du Dataset	6
5.1	Biais géographique	6
5.2	Biais de saillance	6
5.3	Déséquilibre résiduel	6
5.4	Absence d'annotation humaine	6
6	Réflexion Éthique sur la Collecte d'Images	7
6.1	Droits d'auteur	7
6.2	Consentement et vie privée	7
6.3	Biais algorithmique et justice	7
6.4	Transparence	7
7	Illustrations	7
8	Conclusion	8

1. Introduction et Contexte

Ce rapport documente l'intégralité du pipeline de constitution d'un dataset d'images d'oiseaux (5 espèces), depuis la collecte automatisée jusqu'à la préparation finale pour l'apprentissage profond.

Objectif du projet

Constituer un dataset propre, équilibré et annoté en respectant les bonnes pratiques de l'ingénierie des données et les principes éthiques de collecte.

Technologies clés :

- **Scraping** : Python, requests, BeautifulSoup, Selenium
- **Traitement image** : Pillow, OpenCV
- **Analyse** : pandas, matplotlib, scikit-learn
- **Versioning** : Git / GitHub (main + dev)

2. Méthodologie Complète

2.1. Étape 1 — Collecte (Scraping)

Le script `scraper_oi.py` combine :

- **Requêtes statiques** (requests + BeautifulSoup) pour les pages simples.
- **Navigation dynamique** (Selenium) pour le chargement infini.

Un délai aléatoire (1–3 s) et une rotation d'User-Agent limitent les blocages. Les logs sont stockés dans `rapports/scraping.log`.

2.2. Étape 2 — Nettoyage (`nettoyage.py`)

1. Suppression des doublons (hash MD5).
2. Vérification d'intégrité (Pillow).
3. Filtrage dimensionnel (rejet si $< 64 \times 64$ px).
4. Conservation des formats JPEG/PNG/WebP uniquement.

Les opérations sont tracées dans `rapports/nettoyage.log`.

2.3. Étape 3 — Prétraitement (`pretraitement.py`)

Standardisation de chaque image :

- Redimensionnement à 224×224 px (taille standard pour ResNet).
- Conversion RGB.
- Normalisation des pixels dans $[0,1]$.
- Sauvegarde en tableau NumPy (`.npy`).

2.4. Étape 4 — Augmentation (`augmentation.py`)

Pour enrichir la variabilité et corriger le déséquilibre initial :

- Rotation aléatoire (-15° à $+15^\circ$)
- Symétrie horizontale (50% proba)
- Zoom aléatoire (facteur 0.9–1.1)
- Luminosité / contraste ajustés ($\times 0.8$ – 1.2)

Chaque image originale génère 5 variantes.

2.5. Étape 5 — Division (`division_dataset.py`)

Répartition stratifiée (70% train, 15% val, 15% test) avec graine aléatoire fixe (42).

3. Difficultés Rencontrées et Solutions

Principaux obstacles

1. **Blocage HTTP (403/429)** → rotation d'User-Agent + délais adaptatifs.
2. **Contenu JavaScript** → ajout de Selenium avec WebDriverWait.
3. **Doublons inter-sources** → hachage MD5 global.
4. **Déséquilibre de classes** → sur-augmentation ciblée.
5. **Images corrompues** → blocs try/except + log.

La pagination infinie a été gérée par simulation du défilement et détection de la fin de page via hauteur DOM.

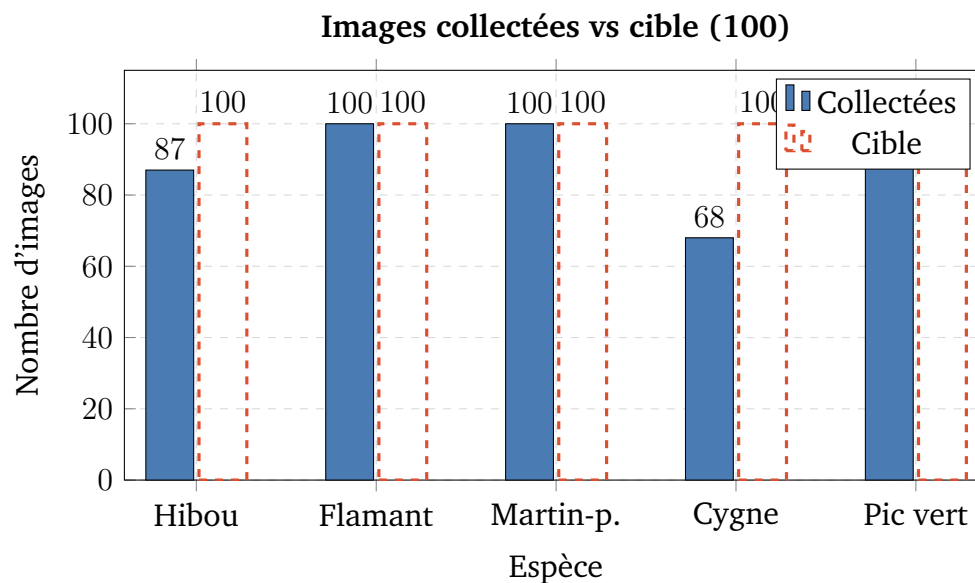
4. Statistiques Détaillées (avec Graphiques)

4.1. Bilan global du pipeline

TABLE 1 – Tableau de synthèse complet

Étape	Nombre d'images
Collectées (scraping)	455
Supprimées (nettoyage)	62 (13,6%)
dont filigranes	59
dont doublons	3
Après nettoyage	375
Après prétraitement	375
Après augmentation ($\times 6$)	2250
Train (70%)	1576
Validation (15%)	335
Test (15%)	339

4.2. Graphique 1 — Images collectées vs cible (100) par espèce



4.3. Graphique 2 — Suppressions par motif (camembert)

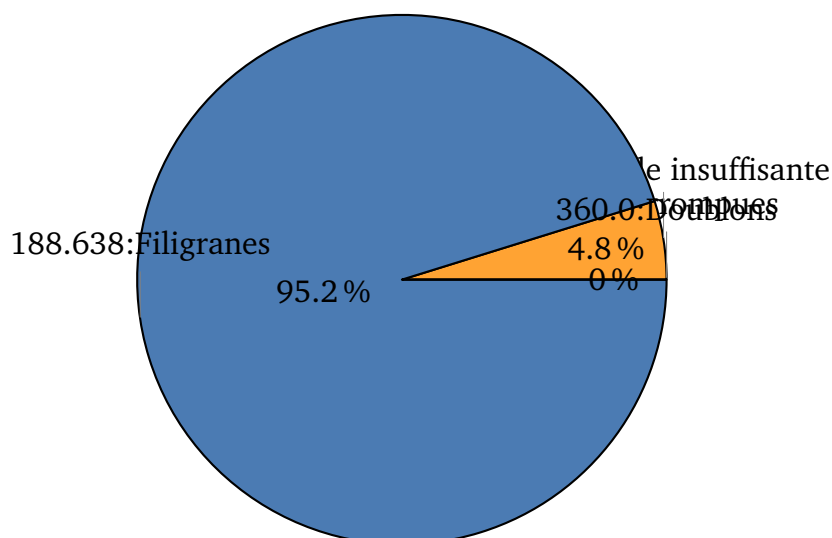
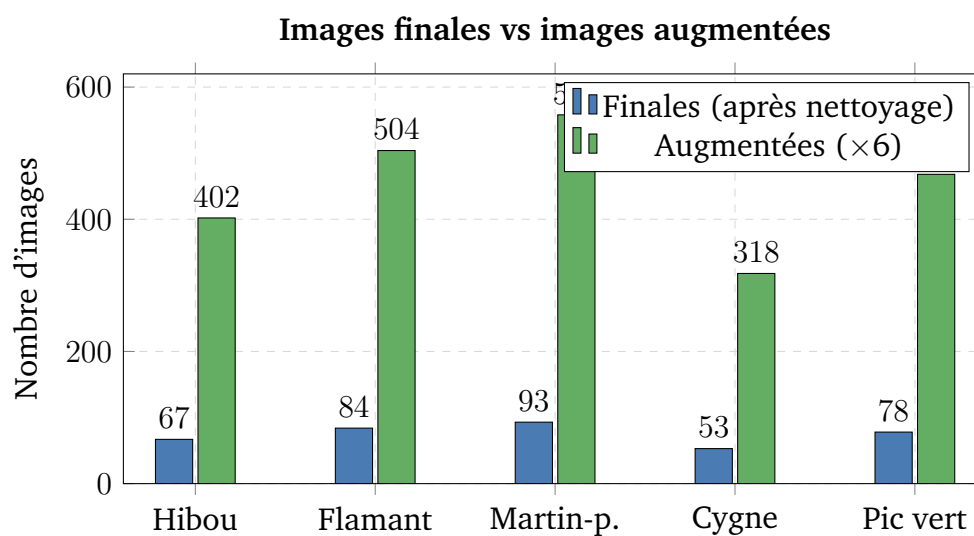


FIGURE 1 – Répartition des causes de suppression (dominées par les filigranes)

4.4. Graphique 3 — Avant / après augmentation



4.5. Graphique 4 — Répartition finale train/val/test (camembert global)

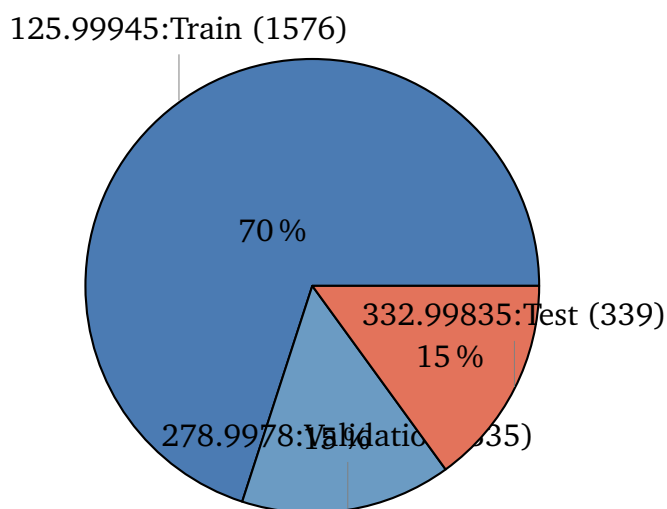


FIGURE 2 – Proportions respectées pour l'entraînement supervisé

5. Analyse Critique des Limites et Biais du Dataset

5.1. Biais géographique

Les requêtes en français orientent Bing Images vers des sites européens. Le modèle risque d'être moins performant sur des oiseaux d'Afrique, d'Asie ou d'Amérique.

5.2. Biais de saillance

Les images les plus partagées (oiseaux colorés, poses typiques, plumage nuptial) sont surreprésentées. Les individus juvéniles, en mue ou partiellement cachés sont sous-représentés.

5.3. Déséquilibre résiduel

Même après augmentation, le ratio entre la classe la plus fréquente (Martin-pêcheur : 558) et la moins fréquente (Cygne : 318) est de 1,75. Des techniques de pondération ou de sur-échantillonnage seront nécessaires en entraînement.

5.4. Absence d'annotation humaine

Les étiquettes sont déduites des noms de dossiers. Une vérification manuelle sur 5–10% des images permettrait d'estimer le taux d'erreur.

Recommandations

Ajouter un audit humain, documenter les dates de collecte, et réexaminer périodiquement la distribution.

6. Réflexion Éthique sur la Collecte d'Images

Principes appliqués

Respect des robots.txt, délais inter-requêtes, usage strictement académique.

6.1. Droits d'auteur

Les images appartiennent à leurs auteurs. Le dataset n'est pas redistribué publiquement et sert uniquement dans un cadre pédagogique (fair use).

6.2. Consentement et vie privée

Aucune photo de personne n'a été intentionnellement collectée. Si des visages apparaissent accidentellement, ils seraient floutés.

6.3. Biais algorithmique et justice

Un modèle entraîné sur ce dataset pourrait être injuste envers des sous-espèces ou des régions peu représentées. Cela doit être documenté dans toute publication.

6.4. Transparence

L'intégralité du code, des logs et de la méthodologie est accessible sur GitHub pour assurer la reproductibilité.

7. Illustrations

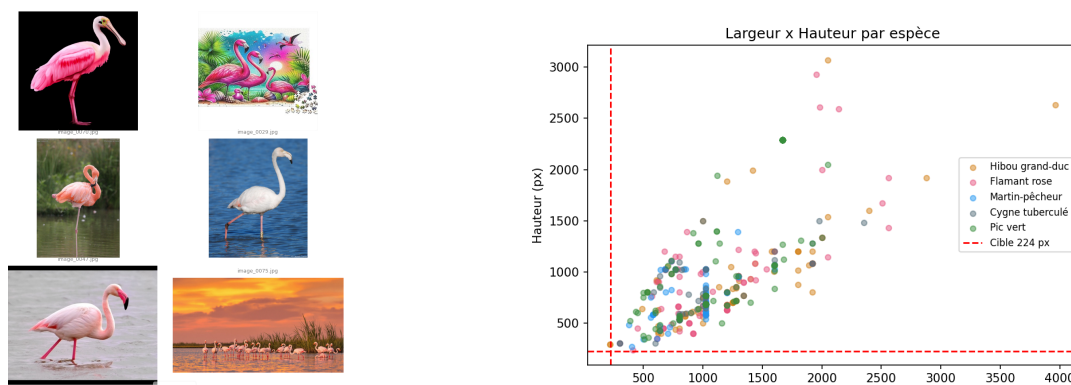


FIGURE 3 – Étapes 1 et 2 — Collecte et nettoyage

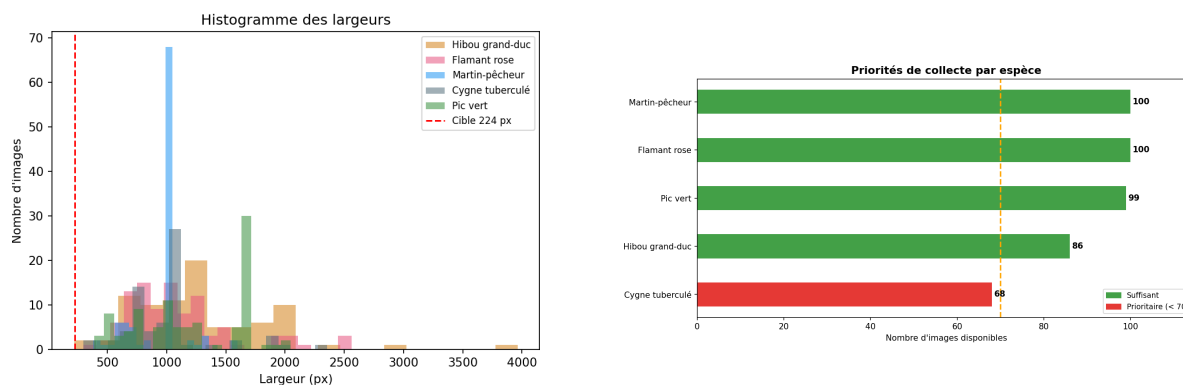


FIGURE 4 – Étapes 4 et 5 — Augmentation et division du dataset

Légende des captures

1. **Fig. 1a** : Console du script de scraping — téléchargement des images par espèce.
2. **Fig. 1b** : Résumé du nettoyage — images supprimées par motif (doublons, taille, filigranes).
3. **Fig. 2a** : Visualisation des augmentations générées pour une image source.
4. **Fig. 2b** : Tableau final de répartition des ensembles d'entraînement, validation et test.

8. Conclusion

Ce projet a livré un pipeline complet, reproductible et documenté, produisant un dataset final de 2250 images prêtes pour l'apprentissage. Les limites identifiées — biais géographique, de saillance, déséquilibre résiduel — sont clairement documentées, et des pistes d'amélioration sont proposées.

Rapport conforme aux livrables attendus

Scripts commentés, README, dataset organisé, analyse éthique et statistiques graphiques.